

# Embedded System Laboratory: Vision in loop demo

**Damian Gaethofs**  
s1234567

**Diego Rooijackers**  
s2759160

2026-07-03

## Contents

|                                                      |    |
|------------------------------------------------------|----|
| 1. Introduction .....                                | 3  |
| 2. Design Space Exploration .....                    | 3  |
| 2.1. Hardware choice .....                           | 3  |
| 2.1.1. Development speed .....                       | 3  |
| 2.1.2. Documentation .....                           | 4  |
| 2.1.3. CPU Power .....                               | 4  |
| 2.1.4. FPGA Performance .....                        | 4  |
| 2.2. Software choice .....                           | 4  |
| 2.2.1. PWM modulation & quad decoder .....           | 4  |
| 2.2.2. Controller performance .....                  | 5  |
| 2.2.3. Communication .....                           | 5  |
| 2.2.4. Image processing .....                        | 5  |
| 2.3. Image processing software specifics .....       | 5  |
| 2.3.1. Development speed .....                       | 6  |
| 2.3.2. Extensibility .....                           | 6  |
| 2.3.3. Complexity .....                              | 6  |
| 2.3.4. Performance .....                             | 6  |
| 2.4. Final design .....                              | 6  |
| 3. Realization .....                                 | 7  |
| 3.1. Quadrature decoders .....                       | 7  |
| 3.2. PWM Module .....                                | 8  |
| 3.3. First time using Avalon bus .....               | 9  |
| 3.4. Extending SPI .....                             | 9  |
| 3.5. Homing procedure .....                          | 10 |
| 3.6. 20-sim Controller Tweaks .....                  | 11 |
| 3.7. GStreamer and OpenCV .....                      | 12 |
| 3.8. Combining vision code with the controller ..... | 13 |
| 4. Evaluation .....                                  | 14 |

|                                          |    |
|------------------------------------------|----|
| 4.1. Vision to controller issues .....   | 14 |
| 4.2. General conclusion on labwork ..... | 14 |
| 4.3. Usage of DSE in ESL .....           | 14 |
| 5. AI Statement .....                    | 15 |

# 1. Introduction

This is our report of the vision in loop demo project done for the course Embedded Systems Laboratory.

First we will start with the design space exploration of our final design. Here we will cover various design decisions, such as which hardware platform to use, what platform is better for which software and how we do our image processing. After our design space exploration, we look at our realization. This includes solutions to the assignment from the assignment manual and how that applies to the final design. We show test benches where applicable and record any issues we ran into.

Lastly, we look back at the project in our evaluation. We discuss the final state of the project and which parts work well and which parts do not work. We also look back on the usage of DSE in this project and what it meant for our final design.

## 2. Design Space Exploration

For our demonstration, we kept it simple, and wanted to do spot tracking of a green ball. This means that the center of mass of the green ball should be in the middle of the camera. The system should respond quickly, and have enough resources to perform other tasks on the side.

To get to this design, the main choice was whether to use the Raspberry Pi or the DE10-Nano. We first consider the difference in hardware between the two, then we take a look at different ways to implement the required software. We also discuss how the image processing should be implemented.

### 2.1. Hardware choice

Starting with the hardware choice, like mentioned we either use a Raspberry Pi or a DE10-Nano. We came up with 4 criteria and graded them based on the implementation, see Table 1.

| Criteria          | Weight | Implementation |           |
|-------------------|--------|----------------|-----------|
|                   |        | Raspberry Pi   | DE10-Nano |
| Development speed | 3      | +/-            | +         |
| Documentation     | 2      | +              | +         |
| CPU Power         | 2      | +              | -         |
| FPGA Performance  | 1      | -              | ++        |
| Total score       |        | 3              | 5         |

Table 1: Hardware trade-off table, where  $- = -2$ ,  $- = -1$ ,  $+/- = 0$ ,  $+$  = 1 and  $++ = 2$

#### 2.1.1. Development speed

Development speed is an important metric for this project, as our time was very limited. Both boards run a Linux OS so the development environment for both is similar. DE10-Nano requires us to use Quartus for compiling verilog code, this is quite a lot slower than the software suite we use for the icoboard. However, we already made both the quadrature encoder and PWM work on the DE10-Nano, so the development time there is marginal.

We are more familiar with SPI than with the Avalon bus, so this would be easier to work with, but we already have a 32 bit Avalon bus implementation, while SPI would have to be extended to 32 bit. The C code is mostly the same for both implementations, only the communication between the FPGA and Linux is different.

In general, while the development speed is very important, the difference between using the Raspberry Pi and the DE10-Nano is large. DE10-Nano has the advantage of not having to implement the communication bus further.

### 2.1.2. Documentation

The documentation is an important part when it comes to debugging issues during development. Raspberry PI is a widely used platform, so a lot of tutorials and documentation can be found for this. The IcoBoard however is a bit more obscure, but is open source and still has a decent amount of information available.

The DE10 is less widely used than the Raspberry PI, but there are lots of documents written by the company who developed them. There is also extensive documentation on the Avalon bus, though it can be a bit much at times.

Both boards are running the Linux Kernel, for which there is plenty of documentation. Both toolchains seem to have plenty of documentation.

### 2.1.3. CPU Power

The Raspberry Pi 4 contains a Quad core cortex A-72 which runs at 1.8 GHz. The DE10-Nano runs a dual core cortex A-9 at 800 MHz. The raspberry pi is obviously more powerful than the DE10-Nano in terms of cpu power. However, it is hard to say if the power of the raspberry pi is actually necessary for the tasks we want to perform. Quite likely, the processor on the DE10 is more than powerful enough to run the full vision in loop demo. Of course, the raspberry pi will have more room for further enhancements in case of future extensions, which can arguably make it the better platform.

### 2.1.4. FPGA Performance

There is a big difference in the performance between the icoBoard FPGA and the DE10-Nano FPGA. The icoBoard has a clock of maximum 100MHz while the DE10-Nano has a clock of maximum 50MHz. However, the icoBoard has 8000 logic elements while the DE10-Nano has almost 110000 logic elements. This means the DE10-Nano is able to support a lot more designs, which makes it the best choice for FPGA performance.

## 2.2. Software choice

| Criteria               | Weight | Implementation |           |
|------------------------|--------|----------------|-----------|
|                        |        | Rpi/ICO        | DE10-Nano |
| PWM modulation         | 2      | +              | +         |
| Quad decoder           | 2      | +              | +         |
| Controller performance | 1      | +              | +         |
| Communication          | 3      | -              | ++        |
| Image processing       | 2      | ++             | +         |
| Total score            |        | 6              | 13        |

Table 2: Software trade-off table, where  $- = -2$ ,  $- = -1$ ,  $+/- = 0$ ,  $+ = 1$  and  $++ = 2$

The eventual goal of the setup is to achieve high speed reference tracking. As mentioned before, implementation time is also limited. A setup has to be chosen which is both speedy, easy to implement, and, additionally, not prone to error.

### 2.2.1. PWM modulation & quad decoder

In order to achieve the speed requirement, PWM commands should be performed quickly, and the quadrature decoders should be read out on time and correctly. Both the Rpi/ICO and DE10-Nano setups directly apply the code directly to their respective FPGA's, effectively making each code run as fast as direct hardware.

### **2.2.2. Controller performance**

In terms of speed, the controller works on a certain frequency. The period is longer than the runtime of the controller. Hence, timing would be less of an issue resulting in the low weight.

Both setups run on a C++ code generated from 20sim. Timing seems to be not a noticeable issue for either chip, hence both receive the same score.

### **2.2.3. Communication**

The biggest bottleneck of the system was identified to be the communication protocol between the ARM chip and the FPGA. A bus should deliver fast, accurate and consistent data.

The DE10-Nano uses the Avalon Bus as the main communication protocol. This is a parallel on-chip communication protocol which uses a memory-mapped interface. This nature allows for high bandwidth communication and low latency, as memory is directly read out or written to. In the verilog script, little code has to be written for the communication, since most of the bus is set up in Quartus.

A downside on using the Avalon bus is the use of proprietary software. This makes us dependent on using Quartus. We also lose the ability to fully understand how communication happens between both chips.

The Rpi/ICO setup uses an SPI bus. This is a widely known serial communication protocol. It can also be read out, allowing for full transparency of the bus.

As mentioned before, speed is very important. SPI's serial nature is quite slow compared to directly reading out and writing to memory. It also offers worse bandwidth. Additionally, timing would become an issue as communication is very precise. This makes it more difficult to program and more easily prone to human error. Since speed is deemed of most importance, this setup is heavily penalized.

Since we do not know how the Avalon bus works, we cannot fully compare the use of resources, like bit length. We do assume that due to the parallel nature, the Avalon bus would be significantly faster than the not-so-quite resourceful serial SPI bus.

### **2.2.4. Image processing**

Image processing could take up quite some resources as well as time in order to determine to correct homing location. It is important the CPU receives the images quickly and processes them accurately and quickly.

The ARM chip on the DE10-Nano runs a fully operational Linux kernel . It is therefor perfectly capable of running the GStreamer pipeline, and decode the images.

The Raspberry Pi is a highly capable single-board computer. It is known for having excellent support of multimedia and the GStreamer framework. It can handle any format, like MJPEG or YUV with ease. This makes it edge out the DE10-Nano setup.

## **2.3. Image processing software specifics**

Image processing can be done on both setups, but how the image processing is actually implemented remains to be analyzed. There are several methods for performing the image processing 1) only using GStreamer, processing the frames using our custom functions or 2) using GStreamer together with the OpenCV library 3) using GStreamer + FPGA to have the processing done on the FPGA.

| Criteria          | Weight | Implementation |                  |                |
|-------------------|--------|----------------|------------------|----------------|
|                   |        | GStreamer only | GStreamer+OpenCV | GStreamer+FPGA |
| Development speed | 3      | +              | ++               | -              |
| Extensibility     | 1      | +              | ++               | +/-            |
| Complexity        | 2      | +              | +/-              | +/-            |
| Performance       | 2      | +/-            | +/-              | ++             |
| Total score       |        | 6              | 8                | -1             |

Table 3: Software trade-off table, where  $-- = -2$ ,  $- = -1$ ,  $+/- = 0$ ,  $+ = 1$  and  $++ = 2$

### 2.3.1. Development speed

As mentioned before, development speed is a very important metric for this project, as we have limited time to complete the design. Comparing the three options for image processing, using GStreamer together with OpenCV is expected to go the fastest for us, as we already have experience using OpenCV and have lots of example to draw from. If we were to use GStreamer only, we would have to do a lot of the image processing manually, which we would expect to take longer. Lastly, we could use GStreamer to pass the image data to the FPGA and do the image processing on the FPGA. While the FPGA would do this quite efficiently, the time it would take us to implement this on the FPGA would be far too high to get done on time.

### 2.3.2. Extensibility

A less important metric for this project is how easily extensible the system is. OpenCV is used a lot in the industry and has a lot of support from there, so extending the system would be quite simple. On the other hand, using only GStreamer or using the FPGA for image processing means we have to write all imaging processing functions ourselves, which makes any extensions a lot of work.

### 2.3.3. Complexity

Now for complexity, using only GStreamer is the simplest option, because there are no extra libraries involved, however that does mean we have to write the image processing functions ourselves. OpenCV comes with a lot more complexity from the library itself, increasing the overall size of the project. Performing the image processing on the FPGA means writing all the image processing code in Verilog, which is quite a complex task, and not very feasible for this project.

### 2.3.4. Performance

Lastly we have performance, we expect that GStreamer and GStreamer + OpenCV have similar performance, OpenCV might be slightly slower due to overhead. We expect the FPGA to be the highest performing, since it basically runs in hardware, making it very fast.

## 2.4. Final design

The final design which follows from the entire design space exploration is going to look as follows. We will use the DE10-Nano as our platform of choice, with the quadrature decoder and the PWM module running on the FPGA. The controller for the motors and the image processing will happen on the ARM chip. Communication between the FPGA and ARM chip will be done using the Avalon bus, with a 32 bit bus.

For image processing, we will use GStreamer to get the camera feed from the USB camera, and send that data to our OpenCV software. OpenCV will calculate the normalized values for the position of the object, which we will use to set the new reference position of the model.

Now that the final design is set, the next chapter will cover the realization of the design.

### 3. Realization

With the DSE done, we will now cover the realization of the DSE. We will cover the different modules that we designed, implemented and tested and how all of it fits together to form the final demo.

#### 3.1. Quadrature decoders

Measuring the position of a motor is an important part for controlling a motor. For this reason motor encoders exist, of which there are a variety of types. The one we use in this project is a quadrature encoder. This type of encoder uses 2 signal lines for encoding. The speed and direction of the motor is encoded onto the signal using the frequency of the signal, and which of the signal lines rises or falls the first. The position can be tracked by counting up and down depending on the direction.

To implement a decoder on an FPGA, we came up with a state machine approach. The current state is made up of the values of each signal line. This results in a 2 bit value, which gives us a state machine with 4 different states. In each state, we check what the previous value was and count up or down depending on the direction. We also set a direction value which we export. This process can be viewed in Figure 1.

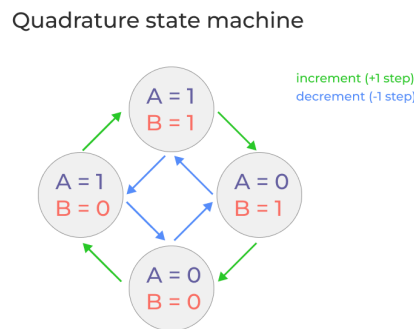


Figure 1: Quadrature state machine design principle. Image taken from: <https://www.fiveflute.com/guide/understanding-quadrature-encoders-a-comprehensive-guide-for-mechanical-engineers/>

In Figure 2, the testbench results of our implementation can be found. The count variable was put to 16 bits, since from our measurements, we found that 8 bits would not be enough to measure the full range of the motors. While we do not need the full 16 bits for the range of the motors, we ease of use, we wanted to stick with standard integer sizes and not using something like 12 bits.

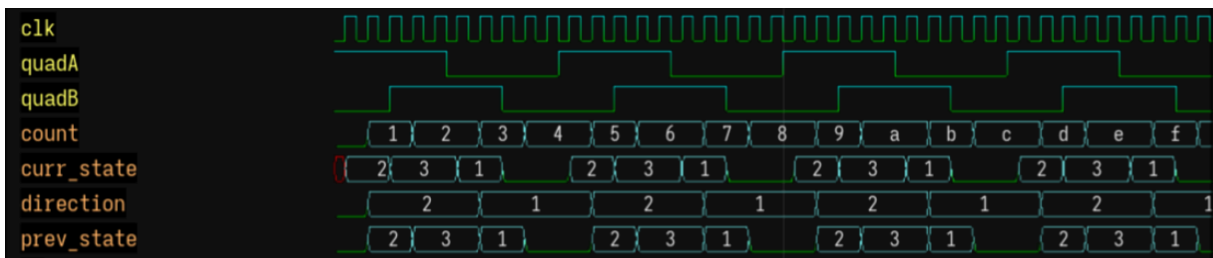


Figure 2: Test bench of quadrature decoder, quadA and quadB are decoder inputs, count is position output

After finishing the decoder, we used it to measure the approximately the encoder count difference for 180 degree angle change. For the tilt angle this was around 9850 counts, and for the pan angle it was around 10850. Using this, we defined a constant for converting encoder count to angle. For example, the tilt constant would be the following:

$$\text{const} = \frac{0.5 \cdot \pi}{9850}$$

We use this in our controller code to convert the raw encoder value read from the bus to the angle in radians required for the model.

### 3.2. PWM Module

With the quadrature decoder done, we still need to be able to control the motors. The motors are controlled through PWM and a direction signal. An IC sets this PWM value into an actual signal and physically controls switches in order to select how the current flows through the motor. A visual of this system is shown in Figure 3.

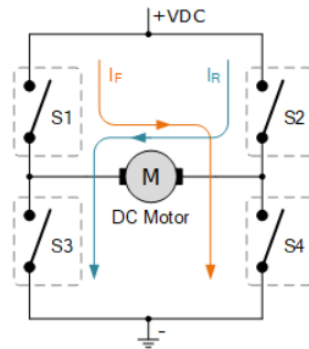


Figure 3: H-bridge configuration of the setup. Image taken from the lab manual.

Our HDL module will need 3 outputs, 2 for the direction and 1 which is the PWM signal. It will need a direction input and a duty cycle input.

PWM operates at a frequency, which we set to be 20 kHz, as that is just outside the human hearing range. To actually operate the PWM value at this frequency, we need to calculate a maximum counter value, which is used to determine when the circuit should switch its state. The counter is calculated as:

$$\frac{50 \text{ MHz}}{20 \text{ kHz}}$$

Where 50 MHz is the clock frequency of the DE10-Nano.

For our duty cycle, we have an 8 bit integer, as input, which we clamp to be between 0-100. It can be noted that this is not effective use of the full range of the available 8 bits, whose value can be up to 255. For cleanliness, it was decided to stick with bytes for as much as possible, as this was easier to implement in messages, in case we wanted to read the values out. To only use values up to 100, effectively percentage PWM using only whole numbers, makes the code more intuitive to use, instead of calculating the percentage into 255 value. This was at a loss of precision, which was deemed insignificant for the given task.

Using the duty cycle, we can calculate a counter threshold. This counter threshold determines at what point we should go from 1 to 0 on our PWM line. If the duty cycle is 100, the counter threshold will be the same as the maximum counter value, meaning the PWM line will always be 1. Anywhere in between means the PWM line goes 0 when the counter exceeds the counter threshold.

The PWM modules then becomes a simple 2 if statement code block. First we check if the counter is not bigger than the absolute maximum value, if it isn't then we add 1 to the counter. We also check if the counter is smaller than the counter threshold and if it is we set the PWM line to 1, otherwise we set it to 0. Lastly, we still have to take care of the direction, we simply take the direction input and then set one of the direction outputs to the direction input, while setting the other direction output to the inverse of the direction input.



Figure 4 shows the testbench results for this module. It starts with duty cycle at 50% (32 hexadecimal), and then goes to 25% in the opposite direction. It shows how the counter threshold changes when the duty cycle changes. The counter is not shown here.

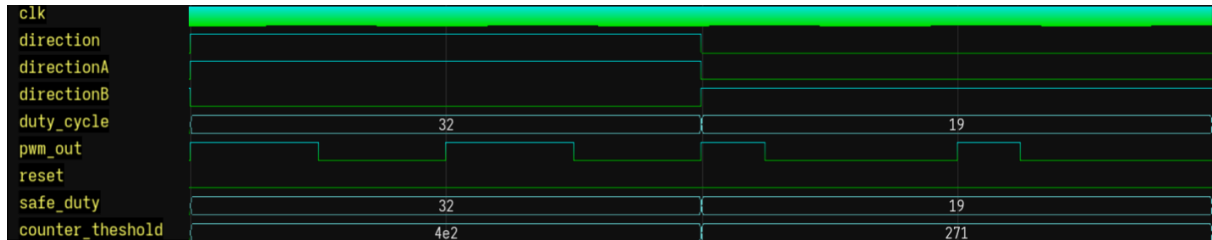


Figure 4: Testbench of PWM module, shows 2 examples, 50% and 25% in opposite direction

We then tested the PWM module by using an oscilloscope on the PWM pins of the PMOD signal measurement board. The oscilloscope reported exactly what we expected, we saw the correct duty cycle and the correct frequency of 20 kHz.

### 3.3. First time using Avalon bus

For assignment 7 we were tasked first with setting up an example project for working with the Avalon bus. We followed the steps and were able to get the example to work quickly. However, when it came to adding the quadrature decoder which we wrote, we started to struggle.

The main issue was: we had no idea what we were doing with the TCL files and what the Avalon bus actually does. While the example showed us the general workflow, it did not tell us anything about how to modify it to get it working with our own design. It also did not really teach us about what was happening in the background, what each generated file contains and how that links with our own stuff. In the end it cost us around 3 sessions just get this working.

The compilation time for making a change in Quartus also did not help in that regards, every change which we made took quite some time to test. This was quite a struggle, but once the Avalon bus was setup correctly, it worked very well. Additionally, a header *soc\_system.h* file was easily generated from quartus for our use case.

The bus was also already set up for our requirements, namely 32 bits for the entire bus. For the communication from FPGA to the ARM-chip, we send two 16 bit unsigned integers which contain the raw encoder values. On the ARM-chip we can easily read these values out using the first 16 bits as a *uint16\_t* and then shifting the bits by 16 and reading another *uint16\_t* to get the second value.

For communication from the ARM-chip to the FPGA, we use a total of 18 bits to control the PWM module. It consists of two 9 bit sequences, with each sequence consisting of an 8 bit duty cycle value and a 1 bit direction value. The direction bit is set depending on whether the output signals from the model are positive or negative.

The application was tested by applying it on the setup and seeing whether we received results. The robot did indeed move based on arbitrary inputs (arbitrarily chosen PWM values). Movement could indeed be read out and printed out in a expected manner (only when moving one side, it the counter would exclusively increase/decrease).

### 3.4. Extending SPI

We also evaluated the possibility the SPI bus in case we wanted to use the RPI. As we were already familiar with SPI and how to use it, it would have been clearer than the Avalon bus. However, the SPI as implemented in assignment 10 and 11 is a single byte SPI bus. For our design we wanted to have at least a 32 bit bus, to be able to send the data as we do in the Avalon bus. So extending the SPI bus to 32 bits was required.

We started our extension with attempting to set the bits per word in SPIdev to something higher than 8, this however turned out to not work. The SPIdev kernel module did not seem to work in this case, so instead we switched to changing the length option, which specifies the amount of bytes we transfer. Our first thought was that changing the length to 4 means the SPIdev module sends 4 bytes in succession with chip select going back high after each bit. Analyzing the bus with the logic analyzer showed that this was not the case, chip select always stayed low. This means the only thing we would have to do to get the FPGA implementation working with 32 bits was change every register and counter to assume 32 bits.

Sounded simple, but after changing this, we were unable to get it working. The FPGA would never send any data correctly over the bus, occasionally it would send a single bit at the end of the transfer, but nothing useful. We asked TAs but never figured out how to get it working, so we stuck with the DE10 and the Avalon bus. This part forms the basis of part of the eventual DSE.

### **3.5. Homing procedure**

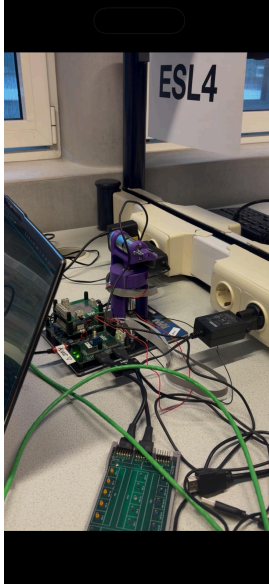
One of the issues we had to deal with was encoder value overflow. We used our homing sequence to fix this. Our homing sequence consists of two movements, first it moves to the minimum value, stores that and then moves to the maximum value. We use the minimum value as an offset which we subtract from the encoder value. This means our raw position data, which would previously overflow, is now limited to a range which fits in 16 bits.

The actual implementation for homing was done very naively, we simply have the motors move in one direction for 3 seconds with a PWM value of 10 per cent. We found this slow enough that the system does not become unstable, yet fast enough for quick homing. We assume it has reached end point and then use the measured encoder value as the corresponding endpoint. We see that a more sophisticated approach would be to measure the change in encoder value, stopping homing when the value stops changing, or rate of change falls underneath some threshold. However, the naive approach works reliably for this set up and was quick to implement. Hence, we decided to keep this due to lack of time.

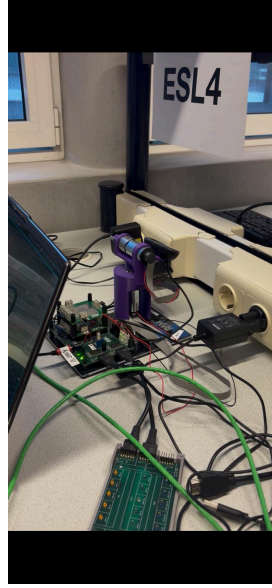
The minimum and maximum values of the pan and tilt range are then stored. The initial setpoint, where the robot will rest before input has been given, is then set to the center of both tilt and pan based on these maxima.

Additionally, a little bit of raw slack is given to the found maxima. This was added because we found the actual max value could slightly alter during run time, like if a max raw value of 200 was found initially, it could become 202 when measured during runtime again. The slack is set to 100, as this seemed to give the most consistent results. It could be tuned lower in order to have the values become closer to reality, but due to timing we were unable to do so.

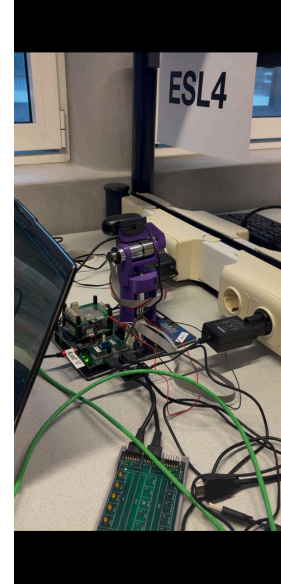
The results of the homing procedure can be found in Figure 5. It can be seen that it moves to both sides for pan and tilt each, and settle at the middle point.



(a) Minimum position



(b) Maximum position

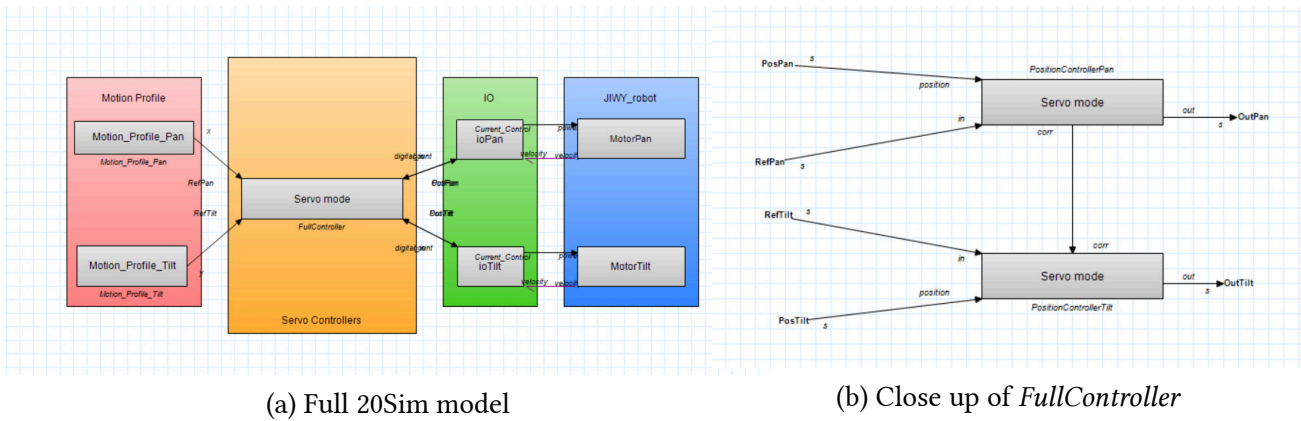


(c) Middle position

Figure 5: Positions found during homing procedure.

### 3.6. 20-sim Controller Tweaks

The presented 20sim model provided a two connected PID controllers: one controller for pan and the other for tilt. The Pan output include a variable which is then used for the tilt input. This is assumably a correction factor used because the two systems are interconnected, linking the effect of the pan to tilt within the controller. Implementing this controller as it was presented, seemed to lead to questions about how to correctly implement this interconnection in the c++ code. To solve this, we merged the two controllers into a single controller block, this can be seen from Figure 6. This ensures that the code generator from 20sim solves these issues, rather than us, which prevents human error and thus possible debugging. We therefore only have to use a single input vector which consist of pan and tilt position and reference. This also meant the only output became the PWM values.

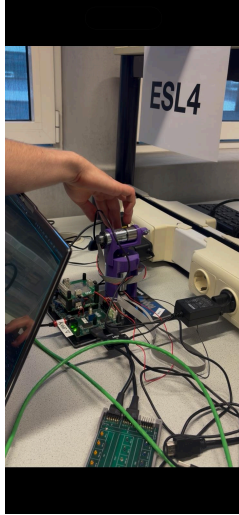


(a) Full 20Sim model

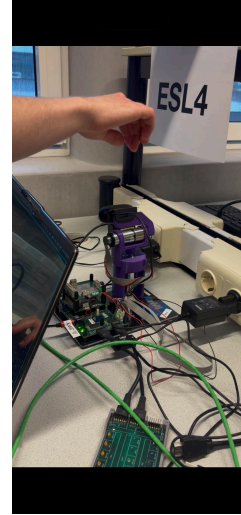
(b) Close up of *FullController*

Figure 6: 20sim model.

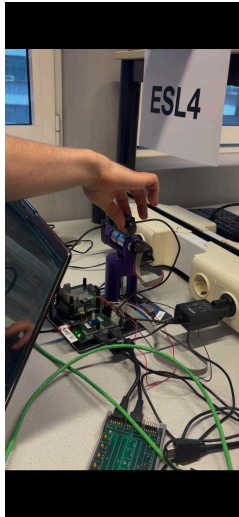
The *FullController*-block was used to generate c++ code to implement on the system. As mentioned before, the initial reference is set to the calculated middle of the possible range based on the homing sequence. The controller ensures that this setpoint is respected without a change of reference. To test this, the tilt and pan positions were manually disturbed by putting them into their maximum positions. The controller is supposed to bring them back consistently to the reference. Figure 7 shows the disturbance as well as the resulting return to reference. This shows the correct workings of the controller.



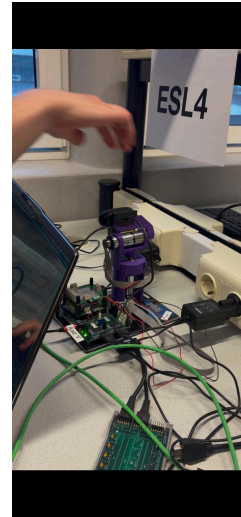
(a) Pan placed out of position



(b) Return to pan reference



(c) Tilt placed out of position



(d) Return to tilt reference

Figure 7: Testing the controller by displacing the system to a extreme of the range.

### 3.7. GStreamer and OpenCV

We used GStreamer and OpenCV to apply the image capture and processing, respectively. Firstly, we created a video pipeline using GStreamer. The pipeline starts by opening the camera, which it find through `v4l2` command. We would like to have both a image capture as well as a image stream. Hence, we created a *tee*-element to split the pipeline into a capture line and stream. The source line encodes and decodes the video capture into and from jpeg to ensure the data arrives uncorrupted and usable for processing. The video feed is then fed to a buffer, which is then fed through to our OpenCV program. In this process, the frame is transformed to a *mat*, which is a OpenCV type used to do image processing. On the side, a sink element is created to show the image feed parallel to the video capture.

On the openCV side, a class was created to handle the image processing. The main program has a *Mat*-frame as input. It checks that the frame is not empty, otherwise the program returns. It converts the image from BGR to HSV, as this color scheme is more robust against different lightning of the objects. These values are then used to filter out all pixels that fall within the bounds. These bounds were found using a online tool (<https://pseudopencv.site/utilities/hsvcolormask/>) and a test image. A masked frame is created, effectively making all left over pixels white and the rest black, to make the following steps easier. An additional layer of processing is applied, which first get rid of background

noise (white pixels outside the main object) and fills in holes (black pixels in the main object). This was achieved by using the *morphologyEx*-function of OpenCV with a rectangular kernel. Moments can be found using the *Moments*-function, and the actual center of gravity (CoG) of the image can be calculated as follows:

$$\text{CoG}_x = \frac{M_{10}}{M_{00}}$$

$$\text{CoG}_y = \frac{M_{01}}{M_{00}}$$

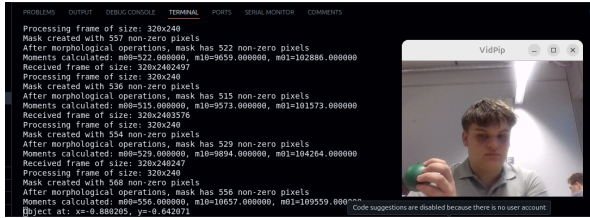
Eventually, we would like the robot to move to the position of the green object. Hence, it would be wise to find the position in a normalized form. It was chosen to have the center of the camera be the center (0,0). The right-up corner would be (1,1) and the left bottom corner (-1,-1). This allows a easier calibration in the controller later on, as we only consider values within a small range of values. These values are calculated as follows, using the columns and rows (width and height) of the frame:

$$\text{CoG}_{x_n} = 2 \frac{\text{CoG}_x}{\text{columns}}$$

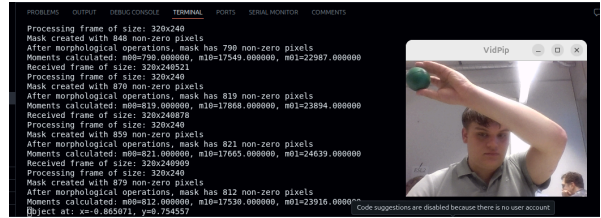
$$\text{CoG}_{y_n} = 1 - 2 \frac{\text{CoG}_y}{\text{rows}}$$

The function outputs these normalized values as a *tuple*-type.

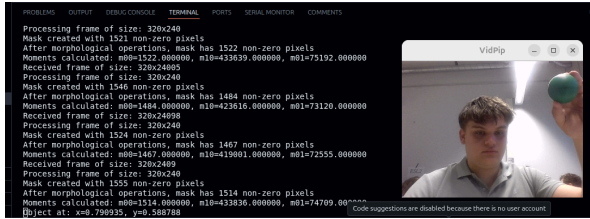
This was tested using one of our laptop cameras and a green ball, as can be viewed from Figure 8. The ball was tracked well to the correct corners. The background was decently noisy, yet the position was always tracked to the ball.



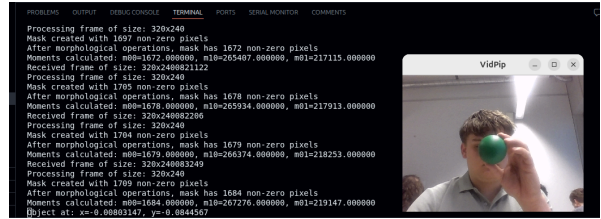
(a) Ball at left bottom corner



(b) Ball at left top corner



(c) Ball at right top corner



(d) Ball at approximate center

Figure 8: Green ball tracking and the image processing normalized CoG output.

### 3.8. Combining vision code with the controller

Now that we have both the controller working and the GStreamer application combined with OpenCV to detect the object, we still needed to combine these. This comes with a few issues, for one GStreamer applications use their own loop with threads made and controlled by GStreamer. It was not as simple as just calling a GStreamer function to run the computer vision aspect; we had to add our controller as a separate thread. To do this we converted our Controller into a class and used `std::thread` to make a

new thread of our controller loop. This is done within the main function of the GStreamer application, making that our main function.

That was not the only issue however, we also needed to get data from the vision part to the controller. To do that, we had the function which processes the frames return the normalized x and y values of the object as two doubles. We could then call a function of the Controller class which would set the references for the model. This function does some additional scaling based on the system performance, to ensure the reference is a functional value for the model.

With that, all the parts of the demo are done, however we did not get it working on time, the next chapter will discuss some of the reasons why.

## **4. Evaluation**

In this chapter we will shortly evaluate how the project went what parts are not working yet.

### **4.1. Vision to controller issues**

While we have all the code written, we were not able to get the full demo working. We have both the controller and the object detecting working separately, but we were unable to get the both of them working together on time. In our last session we ran into issues with the controller (which was previously working) suddenly not working anymore. After some debugging we fixed some value conversions and it worked again, but by that point we had very little time left to test the system as a whole. In the end we ran into other issues, like a segfault (which by default gives very little debugging information) which prevented us from getting the whole demo working on time.

We suspected that the last issues had their roots in the threading. GStreamer creates its own thread which requires quite some resources. Meanwhile, ideally, we would like to have the controller and image processing running on separate threads as well for speed. This requires good communication between all threads, since variables are exchanged between them. This is our suggested main cause of the segmentation fault.

### **4.2. General conclusion on labwork**

Besides the unfortunate integration issues, all building blocks did work as expected. The verilog scripts delivered quick, expected and safe results. The controller was proven to stabilize the system for a given reference. The image processing works as expected and intended.

We did find a feedback point for ourselves. The lack of a general structure/framework to work with eventually could have made the code cleaner and more readable. This would be great quality of life improvement for the code.

### **4.3. Usage of DSE in ESL**

The methods introduced for the design space exploration came in handy on deciding what system to use and what architecture. We found it important to quickly come up with the parameters we would like to test the system on, like speed, cost, etc. This is such that we design for these parameters during prototyping, allowing for a fair and balanced decision making.

Additionally, optimization possibilities could also be a factor that we should consider. For example: if we were to absolutely optimize everything, what would be the best outcome. This is because we only prototype for a small period of time. If we had more time, this may actually significantly influence the outcome. One place where this retrospectively could have made a difference is the decision on image processing architecture. If we were to optimize everything, we could have done everything using GStreamer, which would be absolutely the fastest if done correctly.



In terms of DSE methods applied, we did not try out Pareto Diagram, since we only compare at most 3 systems/architectures. If more options were available, such a schematic could show more information. The method chosen now did give us all the information necessary, and substantiated.

## **5. AI Statement**

During the preparation of this work the author(s) used ChatGPT and Gemini in order to help debugging code. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the work.